



AI OPERATIONAL READINESS & GOVERNANCE

Practitioner Guide

From AI Experimentation to Accountable Operations

A practical operating methodology for discovering, designing, governing, piloting, operating, and improving AI-enabled work.

Version 1.0 | September 2026

Prepared by Chloe C. Devine

How to use this guide

This is the narrative and practitioner layer above the AI Operational Readiness Kit. The guide explains the method and sequence; the existing PowerPoint assets remain in the workshop, working-template, and executive-communication tools.



Publication & Licensing

Copyright © 2026 Chloe C. Devine. All rights reserved except as permitted under the license below.

This work is licensed under the **Creative Commons Attribution–NonCommercial–NoDerivatives 4.0 International License (CC BY-NC-ND 4.0)**.

You may share and redistribute this work in its original form with appropriate attribution. You may not use the work for commercial purposes or distribute modified versions without separate permission from the author.

Version: 1.0

Published: September 2026

Author: Chloe C. Devine

Suggested citation:

Devine, Chloe C. (2026). *AI Operational Readiness & Governance Practitioner Guide: From AI Experimentation to Accountable Operations* (Version 1.0). <https://doi.org/10.5281/zenodo.22911433>

Permanent publication identifier:

<https://doi.org/10.5281/zenodo.22911433>

AI Transparency and Authorship Note

OpenAI ChatGPT was used during the development of this guide as a research, synthesis, editorial, and critical-review assistant. It supported activities such as exploring concepts, comparing sources, testing explanations, organizing material, and reviewing drafts.

The methodology, scope, practitioner judgments, final content decisions, and conclusions are those of Chloe C. Devine. AI-generated suggestions were reviewed, challenged, revised, accepted, or rejected throughout development. The author retains responsibility for the final published work.



Contents

Preface: What This Guide Is For	6
1. The Operating Problem.....	7
The Operational Readiness Gap	7
The Core Practitioner Question	7
2. The Practitioner Method	8
Stage Control Principle.....	8
3. Monday Morning: How to Begin an Engagement	9
First 30 Days: A Practical Sequence	9
Engagement diagnostic vs. use-case design	9
Who to Meet	9
4. Discover the Real Environment	10
Discovery Domains	10
A Better Discovery Technique: Trace a Real Use Case	10
5. Prioritize Opportunities Before Designing Solutions	11
A simple 1-5 scale example:.....	11
Classify the AI Pattern	12
6. Design the AI-Enabled Operating Model	13
Current State	13
Future State.....	13
7. Translate Human Oversight into Decision Rights	14
ACK ≠ AUTH	14
8. Make Governance Observable Through Evidence.....	15
Evidence Under Pressure.	15
Source and context authority	15
Decision Provenance	16
For consequential AI-enabled actions, ask:.....	16
Continued permission is not evidence of continued legitimacy.	16
Temporal Control Separation	17
Decision-basis sufficiency.....	18
9. Use Pilots to Test the Operating Model	19
What the Pilot Should Teach	19
10. Operate: Governance After Deployment	20



From Signal to Governed Action	20
A Governance Operating Rhythm	20
11. Manage Change Before Governance Drifts	21
Material Change Triggers	21
Condition Management.....	21
Three Triggers for Reassessment.....	22
Detecting Representational Non-Fit	22
Designing the Upstream Feedback Path	23
Managing Authority State.....	24
Do Not Force Uncertainty Into Certainty	24
Two Feedback Loops are Required	25
12. Define the Missing Operational Ownership	26
The AI Adoption / Operations Function	26
13. Where Governance Technology Fits	27
From Organization Decision to Runtime Enforcement	27
Verification does not create decision authority.....	28
Pre-commit Control Principle.....	28
Runtime Hold Test	28
Governance Continuity Principle	28
Multi-step agent governance	29
14. Measure Whether the Operating Model Works	29
15. Turn Analysis into Executive Decisions	31
16. Practitioner Conversation Guide	32
With a Business Owner.....	32
With Engineering / Architecture.....	32
With Governance / Legal / Risk / Security.....	33
With Frontline Users and Managers.....	33
17. Operational Red Flags	34
18. Practitioner Deliverables	35
Change Trigger & Reapproval Matrix Example:	35
19. Worked Governance Example: Meeting Summary Assistant	36
20. A Simple Maturity Lens	37
Appendix A. Engagement Checklist	38
Appendix B. Core Principles	38



Appendix C. Development Notes & Practitioner Acknowledgments	39
References	42
Future Research and Validation.....	43



Preface: What This Guide Is For

Organizations increasingly have AI tools, pilots, policies, review processes, and technical controls. What they often lack is a connected operating model that explains how AI-enabled work moves from an idea into everyday operations while preserving human accountability.

This guide is for the practitioner asked to help an organization move from AI experimentation toward responsible, measurable, and durable AI operations. It is not a model-development manual, a regulatory crosswalk, or a substitute for legal, security, privacy, architecture, or compliance expertise. It is an operating methodology for bringing those disciplines together around real work.

The methodology applies whether AI is already in use or is being considered for the first time. For existing AI, discovery reconstructs and evaluates the current operating model. For proposed AI, discovery begins with the existing work and establishes whether AI should play a role at all.

The goal is not more governance documentation. The goal is better operational decisions.

What This Guide Adds to the Original Readiness Kit

The original AI Operational Readiness Kit established a strong path from discovery through pilot readiness. Subsequent practitioner discussion sharpened a second problem: accountability must not only be established before deployment; it must be maintained as systems, workflows, assumptions, data, models, prompts, retrieval sources, vendors, and business conditions change.

- Evidence must remain connected to the system and decision it is intended to support.
- Evidence integrity matters in addition to evidence retention.
- Approval is conditional: when material conditions change, prior decisions may need reassessment.
- Human involvement is not enough; authority, verification responsibility, escalation, and intervention rights must be explicit.
- Monitoring without an owner is not an operating control.
- Governance can drift even when the underlying documentation still exists.
- Pilot readiness must test the operating model, not merely whether the technology works.
- Post-deployment operations need an explicit governance rhythm for monitoring, change, incidents, learning, and retirement.



1. The Operating Problem

Enterprise AI adoption is not only a technology decision. It changes how work is performed, how information is used, how decisions are supported, how outputs are reviewed, and who remains accountable when AI is involved.

A policy can state what an organization intends. A platform can enforce, log, route, or monitor selected controls. Neither, by itself, defines the enterprise operating model: who has authority, what evidence is sufficient, what conditions invalidate an approval, what happens when the system changes, who responds to an incident, or who owns the workflow after launch.

Policy is not the operating model. Technology can enable governance; it does not decide the organization's accountability model.

The Operational Readiness Gap

Organizations often have	But still need to define
AI access	Which work should use AI and why
Policies and principles	Operational controls and decision rights
Pilots	Exit criteria, scale criteria, and ownership
Human review	Verification criteria, authority, and accountability
Logs and documentation	Evidence integrity, traceability, and decision relevance
Monitoring capabilities	Monitoring ownership, cadence, thresholds, and response
Technical change processes	Governance change triggers and reapproval rules
Executive sponsorship	Durable operational ownership after launch

The Core Practitioner Question

Do not begin with “What AI tool should we deploy?”

Begin with: What work are we trying to improve, how does it operate today, what role should AI play, what can go wrong, who has authority, what evidence will prove the system is working, and how will accountability be maintained as conditions change?



2. The Practitioner Method

The methodology is a connected progression. Each stage should produce enough clarity for the next stage; it is not intended to be a rigid waterfall. Teams may loop backward when new information, risk, evidence, or operating conditions require redesign.

Stage	Primary Question	Primary Output
1. Discover	What is happening today?	Operational themes and current-state observations
2. Prioritize	Which opportunities deserve attention?	Prioritized opportunity set
3. Classify	What kind of AI capability is required?	Use-case pattern
4. Design	How should the work change?	Future-state workflow
5. Govern	Who decides, intervenes, and remains accountable?	Decision-rights profile
6. Evidence	What must be observed, retained, and verified?	Evidence plan
7. Prepare	Is the operating model ready to test?	Pilot-readiness decision
8. Operate	How will accountability persist after launch?	Governance operating rhythm
9. Change	What changes require reassessment or reapproval?	Change-trigger model
10. Learn	What evidence justifies scale, redesign, pause, or retirement?	Lifecycle decision
11. Communicate	What does leadership need to decide?	Executive briefing

AI Operational Readiness is not a collection of independent frameworks. It is a connected methodology for moving from uncertainty to informed, accountable operations.

Stage Control Principle

For higher-consequence AI-enabled work, each lifecycle stage should have enough control structure to establish what may enter the stage, what conditions must be satisfied before work advances, what evidence is produced, and what happens when the requirements are not met.

A stage should not advance simply because the previous activity was completed; advancement should reflect a defined readiness or governance decision appropriate to the consequence.



3. Monday Morning: How to Begin an Engagement

When an organization asks for help establishing or strengthening AI governance and operations, resist the urge to begin by designing committees, selecting tools, or rewriting policy. The first objective is to understand the actual operating environment.

First 30 Days: A Practical Sequence

Timing	Practitioner Focus	What You Are Trying to Learn
Days 1–5	Orient	Strategy, AI inventory, policies, current pilots, architecture, major stakeholders, known incidents, existing governance bodies
Days 5–10	Listen	How work happens; where teams are already using AI; where pain, shadow use, and ambiguity exist
Days 10–15	Trace	Reconstruct 1–3 existing or in-flight use cases end-to-end, examining how they moved from idea through evaluation, approval, deployment, monitoring, change, and support.
Days 15–20	Map	Current-state workflow, decision rights, evidence, ownership, escalation, continuity, and change controls
Days 20–25	Diagnose	Gaps between stated policy and operational reality; unclear authority; missing evidence; monitoring and ownership gaps
Days 25–30	Recommend	Prioritized improvements, pilot candidates, governance operating rhythm, and immediate decisions required

Engagement diagnostic vs. use-case design

The first 30 days are intended to understand and diagnose the organization’s current operating environment. For existing or in-flight AI use cases, the practitioner reconstructs how work currently moves through evaluation, approval, deployment, monitoring, change, and support. The practitioner is not expected to perform that entire lifecycle during the diagnostic period.

Once opportunities are prioritized, the methodology shifts from diagnosing current reality to designing a specific future state: what should change, what role AI should play, and what must be established before pilot or deployment.

Who to Meet

- Executive or business sponsor
- Business/process owners
- AI/product/program owners
- Engineering and architecture
- Data and analytics
- Security and identity
- Privacy and legal
- Risk, compliance, and Responsible AI
- Procurement/vendor management
- Change management, learning, and communications
- Frontline users and managers
- Service/support/incident-management owners

The operating model is usually distributed across people. Discovery reconstructs it.



4. Discover the Real Environment

Discovery begins with work, not tools. The purpose is to identify friction, understand workflows, evaluate opportunities, surface dependencies and risks, and determine where accountability currently lives.

Discovery Domains

Domain	Questions to Explore
People	Where is time spent? What is repetitive? Where is knowledge trapped in individuals? Which critical decisions, exceptions, system assumptions, operating knowledge, or escalation paths depend on one person knowing what to do?
Process	Where are bottlenecks, duplicate work, handoffs, waiting, manual coordination, and rework?
Technology	What tools and AI are already in use? What data and systems are required? Where is shadow AI occurring?
Governance	Where is human review required? Who approves? Who can intervene? What happens if the work is wrong?
Measurement	What does better mean? What baseline exists? What evidence would justify scaling?
Adoption	Where is trust high or low? What workarounds exist? Has AI reduced effort or shifted it to reviewers?
Continuity & resilience	What happens if the AI, vendor, model, or API is unavailable for 24 hours, seven days, or permanently? If the person who understands this system, workflow, or governance decision becomes unavailable tomorrow, could the organization still operate, explain, govern, and recover the process correctly?
Escalation & response	How are issues identified, preserved, escalated, paused, communicated, and learned from?
Ownership	Who owns the process, outcomes, monitoring, changes, incidents, and cross-functional dependencies?

A Better Discovery Technique: Trace a Real Use Case

Interviews can reveal stated processes. A real use case reveals operational reality. Select an AI-enabled workflow and trace it from beginning to end.

1. Where did the idea originate?
2. How was it evaluated and prioritized?
3. What data, models, vendors, systems, prompts, or retrieval sources does it depend on?
4. What downstream capabilities, tools, agents, permissions, systems, or consequential actions can this AI system reach directly or indirectly?
5. What testing occurred?
6. Who reviewed and approved it, and what exactly did they approve?
7. What evidence supported the decision?
8. Who deployed or enabled it?
9. Who owns the workflow after launch?
10. What is monitored, by whom, and at what cadence?
11. What happens when a material component or business condition changes?
12. What triggers escalation, pause, rollback, or shutdown?
13. What is retained after an incident or decision?



14. How would the organization know the system should be redesigned, scaled, or retired?

5. Prioritize Opportunities Before Designing Solutions

Discovery will usually surface more opportunities than an organization can responsibly pursue. Prioritization balances opportunity drivers against implementation friction.

Opportunity Drivers (+)	Implementation Friction (-)
Frequency	Complexity
Pain level	Risk
AI fit	Continuity risk
Business impact	Change-management burden

A useful prioritization score can combine frequency, pain level, AI fit, and business impact, then subtract complexity, risk, continuity risk, and change burden. The score is a decision aid, not an automatic decision: high-consequence or strategically important opportunities may require judgment beyond a numerical ranking.

A simple 1-5 scale example:

Factor	1	3	5
Pain	Minor inconvenience	Noticeable delay/rework	Major cost, delay, or burden
Frequency	Rare	Weekly	Daily/high-volume
AI fit	Weak fit	Plausible fit	Well-established fit
Business Impact	Low	Moderate	High
Complexity	Simple	Moderate	Highly integrated/ complex
Risk	Low consequence	Moderate	High consequence
Continuity risk	Easy Fallback	Some dependency	Critical dependency
Change burden	Minor	Moderate	Major role/ process change

Then perhaps:

Opportunity Score = Frequency + Pain + AI Fit + Business Impact – Complexity – Risk – Continuity Risk – Change Burden

Customer refund authorization might score high on pain/frequency but lose points for risk and complexity. The key is for the organization to define the scoring rubric before scoring use cases.

Not every opportunity should become an AI project.



Classify the AI Pattern

An AI pattern is the functional way AI is being used within a workflow—for example, search, summarization, extraction, drafting, classification, recommendation, routing, or autonomous action. A single system may contain multiple patterns.

Pattern	Typical Purpose	Governance Implication
Search	Locate information	Source quality, access, traceability
Summarization	Condense information	Omission, context loss, verification
Extraction	Structure information	Accuracy, downstream data quality
Drafting/rewriting	Create or improve content	Review, attribution, external exposure
Classification	Categorize content	Bias, thresholds, error consequences
Analysis/recommendation	Support decisions	Decision consequence, explainability, human judgment
Routing/monitoring	Coordinate workflow	Thresholds, escalation, false positives/negatives
Agentic workflow	Execute multi-step work	Permissions, authority, state, escalation, rollback, cumulative trajectory
Autonomous action	Execute actions with limited intervention	Highest need for bounded authority, runtime controls, monitoring, and intervention



6. Design the AI-Enabled Operating Model

Design AI-enabled work before deployment. The design sequence begins with current reality and ends with a measurable business outcome. For a proposed use case, this stage shifts the practitioner from diagnosing the existing environment to designing how the work should operate with AI before pilot or deployment.

Current State → Pain Point → Opportunity → AI Assistance → Human Oversight → Business Outcome → Resilience Check

Current State

- **Workflow:** how the work is performed today
- **Inputs:** information, documents, systems, and data required
- **Outputs:** artifacts, decisions, reports, or actions produced
- **Dependencies:** people, teams, approvals, systems, vendors
- **Constraints:** pain, delay, review burden, policy restrictions, bottlenecks

Future State

- **AI role:** what the system does and does not do
- **Human handoff:** where AI assistance ends
- **Review and approval:** what humans verify and authorize
- **Escalation:** what conditions require additional attention
- **Fallback:** how work continues if AI is unavailable or unsuitable
- **Business outcome:** what should improve and how it will be measured
- **Reachability boundary:** What tools, agents, systems, data, permissions, and downstream actions can AI access or invoke, and which of those must be explicitly constrained or prohibited?

Design test

If the future-state description says only, “AI generates X and a human reviews it,” the operating model is not finished. Define what the reviewer verifies, what authority the reviewer has, what happens when the reviewer disagrees, what evidence is retained, and who remains accountable.

Agentic design test

For agentic or autonomous workflows, do not define authority only by what the system is instructed to do. Determine what the surrounding architecture makes technically reachable. A reachable capability that isn't intentionally governed can become de facto authority at runtime.

Consequence-bounding test

For agentic or autonomous workflows, do not assume you can identify every possible execution path in advance. Where reachability is uncertain or may change dynamically, define limits on the consequence that an unanticipated path can create. Governance should therefore consider not only what the system can reach, but what it is permitted to change, commit, transfer, trigger, or otherwise cause if an unexpected path becomes reachable.

Practitioner question:

If the system reaches a capability or execution path we did not anticipate, what limits the consequence it can create?



7. Translate Human Oversight into Decision Rights

Human-in-the-loop is not a governance model. Human participation becomes governance only when responsibilities and authority are explicit.

Decision Right	Practitioner Question
Review	Who verifies AI outputs, against what criteria, and how often?
Approval	Who authorizes use, distribution, execution, or exception?
Escalation	What triggers escalation, who receives it, and what response time applies?
Intervention authority	Who can pause, override, shut down, roll back, and restart?
Accountability	Who owns outcomes and consequences?
Scale / retirement	Who decides whether the system expands, redesigns, pauses, or retires?
Authority modification/restoration	Who can narrow, suspend, restore, or expand an AI system's operating authority, and what evidence is required?

Human involvement without decision rights is participation, not governance.

ACK ≠ AUTH

A useful runtime distinction is that successful receipt, authentication, or technical readiness does not by itself prove that an action is currently authorized. For AI-enabled and agentic workflows, authority may depend on role, context, risk tier, current policy, transaction conditions, prior approvals, or other constraints.

Runtime authority question:

Before an AI system or agent acts, can the organization determine not only whether the actor or system is authenticated, but whether this specific action is authorized under the current conditions?

Runtime evidence question:

Before an AI system or agent acts, can the organization determine whether the evidence and context supporting this specific action are sufficiently current, complete, applicable, and reliable to justify proceeding?

Runtime decision-basis question:

For higher-consequence actions, is there sufficient confidence in the evidence, assumptions, reasoning, and objectives supporting the decision to justify proceeding?



8. Make Governance Observable Through Evidence

Governance becomes operational when evidence can support, reconstruct, and test decisions. Design the evidence plan before the pilot produces results.

Evidence Area	Examples
Value	Baseline, cycle time, cost, throughput, quality improvement over time, customer outcome
Quality	Acceptance criteria, correction rate, hallucinations, rework, reviewer findings
Risk	Incidents, sensitive outputs, policy exceptions, threshold breaches, unexpected behavior
Adoption	Usage, satisfaction, workarounds, trust, training need, review burden
Decision	Scale criteria, pause criteria, redesign criteria, retirement criteria, decision rationale
Governance provenance	Intake, risk classification, test results, approvals, conditions, mitigations, owners, versions
Decision provenance	Logging tells you what happened. Provenance helps reconstruct why the action was legitimate.

Without evidence, governance becomes memory.

Evidence Under Pressure.

The next step is not simply retaining evidence. Evidence must remain trustworthy when challenged after an incident, audit, dispute, or material system change.

- **Integrity:** can the evidence be altered after the fact, and if so, is that alteration visible?
- **Provenance:** can the organization identify where the evidence came from and what system states it describes?
- **Traceability:** can a decision be connected to the relevant inputs, tests, approvals, owners, and conditions?
- **Version alignment:** does the evidence describe the model, prompt, retrieval source, workflow, and policy in operation?
- **Decision relevance:** does the evidence still support the decision for which it was originally collected?
- **Decision evidence:** why did we authorize this?
- **Current evidence:** are the conditions supporting that authorization still true?
- **Evidence freshness:** is the evidence that was sufficient to support a decision earlier still sufficient? Can practitioners determine which evidence is time-sensitive or condition-sensitive, what makes it stale, and when refreshed evidence is required for an action, authority, or governance decision to remain valid? (Chatterjee, 2026)

Source and context authority

An AI system can produce a logically coherent output from information that is incomplete, stale, incorrect, conflicting, or inappropriate for the decision being made. The operating model should therefore define which sources and contextual inputs may be treated as authoritative for a given use case, how conflicts among



sources are handled, and how the system or reviewer identifies when the available information is insufficient to support a reliable conclusion.

Practitioners should ask:

- Which sources is the system permitted to rely on?
- Which sources are authoritative for this decision or action?
- How current must those sources be?
- What happens when authoritative sources conflict?
- Can the system distinguish established facts from assumptions, inferred information, or unverified content?
- What conditions should the system use to express uncertainty, seek additional evidence, escalate, or decline to decide?
- How would the organization detect that the system's working representation of the situation no longer matches reality?

Retention is necessary. Integrity, provenance, and continued relevance determine whether retained material functions as evidence.

Decision Provenance

The ability to reconstruct not only what happened, but why a consequential AI-enabled action was considered legitimate at the time it occurred.

For higher consequence use cases, evidence should allow the organization to connect the relevant system state, supporting evidence, applicable policy, authority, approval or override, authorized action, execution, and resulting outcome.

A complete activity log does not necessarily provide decision provenance if those elements cannot be connected into a coherent chain.

For consequential AI-enabled actions, ask:

- What initiated the action?
- What evidence and context supported it?
- Which policy, control, model, prompt, retrieval source, tool, and authority state were in effect?
- What action was proposed?
- What gate, approval condition, or runtime admissibility decision allowed it to proceed?
- Who approved, escalated, granted an exception, or overrode?
- What actually executed?
- What state changed as a result?
- What evidence was preserved?
- Can those elements be reconstructed together later?

A record that proves an action occurred is not necessarily a record that proves why the action was legitimate.

Continued permission is not evidence of continued legitimacy.

At consequential boundaries, an AI system should not proceed solely because it has not detected a



revocation, violation, or stop condition. Where risk warrants it, the operating model should require current evidence that the conditions supporting authority still hold and define when renewed approval or authority is required before execution continues.

Temporal Control Separation

For consequential AI-enabled actions, the evidence trail should distinguish when key governance events occurred rather than collapsing them into a single activity timestamp. At minimum, practitioners should be able to establish:

- when the actor or system was authenticated;
- when the specific action was authorized;
- when the control or enforcement decision occurred; and
- when the supporting evidence was sealed or otherwise made tamper-evident.

These timestamps may differ. Preserving them separately helps establish whether execution relied on current authority or on an approval that had become stale between decision and consequence.

Separation is not only temporal; it is functional. Authentication, authorization, monitoring, enforcement, evidence preservation, and provenance may occur around the same runtime event, but each establishes a different fact. One mechanism should not inherit the governance or evidentiary standing of another merely because they are temporally adjacent.

Occurrence should also remain distinct from enforcement. A control may permit or prevent execution without proving that the consequential action occurred. The evidence layer should establish what was observed about the resulting event rather than treating an enforcement decision as proof of outcome.

For later review, the evidence trail should preserve enough bounded state to reconstruct what policy, authority, evidence, context, and system conditions were in force at the time of the consequential action, without projecting later conditions or interpretations backward onto the event.

In multi-agent or delegated workflows, this becomes especially important at handoffs, where authority may narrow, expire, or become detached from the original approval context.

Separation principle: the executing system should not also be the sole mechanism determining its own legitimacy and preserving the only evidence of that determination.

Governed refusal should produce evidence.

When an AI-enabled action is blocked, held, denied, or prevented from advancing, the operating model should preserve enough evidence to establish what was attempted, which condition was not satisfied, what authority or control prevented execution, and what next action is permitted.

Evidence and judgment are different layers. Evidence infrastructure should preserve and reconstruct what can be established from available observations without automatically deciding whether that evidence is sufficient for a particular governance, audit, legal, or risk conclusion. Different authorities may legitimately apply different decision criteria to the same underlying evidence base. (FactNotebook, 2026)



Decision-basis sufficiency

A governance decision should not rely only on whether the required evidence exists. For higher-consequence actions, practitioners may also need to consider whether the decision rationale, assumptions, objectives, and interpretation applied to that evidence are sufficiently sound for the decision being made.

An action may be properly authorized, satisfy policy requirements, and produce complete evidence while still resting on incomplete context, flawed assumptions, conflicting objectives, or weak reasoning.

Governance should therefore distinguish between:

- **evidence sufficiency** — is the information needed to support the decision available, current, and relevant?
- **decision-basis sufficiency** — are the rationale, assumptions, objectives, and interpretation applied to that information appropriate for the consequence of the decision?

The required level of scrutiny should increase with consequence and risk.



9. Use Pilots to Test the Operating Model

A pilot should validate more than technical feasibility. It should test whether the workflow, controls, evidence, ownership, support, and fallback mechanisms work in practice.

Readiness Area	Before Pilot, Confirm
Business fit	Problem, intended value, sponsor, success hypothesis, priority rationale
Workflow readiness	Current state, future state, AI role, human handoff, fallback
Technical readiness	Tool/model/platform, inputs, integrations, limitations, safe test environment
Governance readiness	Review, approval, escalation, intervention, accountability
Evidence readiness	Baseline, value, quality, risk, adoption, scale criteria
Operational support	Support owner, user guidance, issue reporting, continuity, lessons capture

A pilot should test a defined operating model, not an undefined idea.

What the Pilot Should Teach

- Does the workflow create measurable value?
- Do users trust and use it?
- Are outputs good enough for the intended purpose?
- Is review burden lower, equal to, or greater than the work displaced?
- Do review, approval, escalation, and intervention work as designed?
- What risks appear in practice?
- Are the selected metrics meaningful?
- Were design assumptions correct?
- What changes are required before scale?
- Should the organization scale, redesign, pause, defer, or retire?

Reconstructability is a lifecycle capability.

Before deployment, ask whether existing evidence is sufficient to establish readiness. During operation, periodically test whether the evidence trail can still reconstruct important decisions and actions. After an incident, audit, dispute, or inquiry, determine what the remaining evidence can honestly support.



10. Operate: Governance After Deployment

Deployment is not the end of governance. Once AI becomes part of everyday work, governance must become an operating rhythm.

Operating Responsibility	Questions
Monitoring	What is monitored? At what cadence? What thresholds matter? Who reviews the signal?
Evidence maintenance	What records must remain current? Who owns them?
Issue response	How are incidents and near misses reported, preserved, triaged, and resolved?
Change control	What changes require testing, documentation, review, or reapproval?
Adoption	Are people using the system as designed? Are workarounds or shadow processes emerging – interviews, observations, support tickets, user feedback, usage/telemetry discrepancies, exception records, shadow discovery?
Continuity	Can the workflow continue if the AI capability degrades or disappears?
Ownership	Who owns day-to-day operation, business outcomes, risk, support, governance coordination, and shared governance capabilities used across multiple AI systems?
Lifecycle decision	What evidence justifies scale, redesign, pause, or retirement?

Monitoring without ownership is observation, not control.

From Signal to Governed Action

Monitoring becomes an operating control only when a detected signal has a defined path to accountable action. For material runtime signals, the operating model should establish:

- **Detect** — What condition or event creates the signal?
- **Route** — Where does the signal go, and within what timeframe?
- **Evaluate** — Who determines what the signal means and whether it is material?
- **Decide** — Who has authority to determine the appropriate response?
- **Intervene** — What actions may follow, such as continuing, narrow authority, escalate, pause, roll back, remediate, or shut down?
- **Preserve** — What evidence of the signal, assessment, decision, and resulting action must be retained?

A monitoring capability that can detect a problem but cannot reliably connect that signal to an authorized response is an incomplete operational governance control.

A Governance Operating Rhythm

Cadence	Illustrative Activities
Continuous/event-driven	Threshold alerts, blocked actions, security events, incidents, policy violations
Daily / weekly	Quality review, user issues, escalations, adoption friction, operational exceptions
Monthly	Evidence completeness, performance trends, control effectiveness, open risks, change review
Quarterly	Business value, risk posture, decision-rights effectiveness, ownership, portfolio review
Event-based	Material model/prompt/data/retrieval/workflow/vendor/policy/business-condition change



11. Manage Change Before Governance Drifts

Governance drift occurs when the controls, evidence, assumptions, or approvals for an AI-enabled workflow no longer accurately describe the system and conditions actually operating.

Material Change Triggers

Change	Potential Governance Response
Model or model-version change	Regression testing; quality/risk reassessment; update evidence
System prompt or instruction change	Targeted testing; update version record; reassess behavior
Retrieval source / knowledge-base change	Source-quality review; access review; output testing
New or changed data	Privacy/security/data-quality review; risk reassessment
New user population	Access, training, misuse, and consequence reassessment
New business purpose	Reclassify use case; reassess decision rights and approvals
New geography / jurisdiction	Legal, regulatory, privacy, and policy review
Increased autonomy or tool access	Authority boundaries, permissions, intervention, monitoring review
Changed reachability	New tool, agent, integration, permission, dependency, or downstream capability. Reassess reachable actions, authority boundaries, downstream consequences, intervention points, and monitoring requirements. When reachability cannot be fully enumerated, reassess whether existing consequence limits remain sufficient for newly reachable or unanticipated paths.
Vendor/API/platform change	Technical, security, continuity, contractual, and evidence review
Material incident or near miss	Incident review; control reassessment; possible pause/reapproval
Business/risk-condition change	Revisit assumptions, thresholds, mitigations, and prior approval

Translate these triggers into a Change Trigger & Reapproval Matrix that identifies how each change is detected, who evaluates materiality, what evidence must be refreshed, what happens to operating authority during reassessment, and whether additional review or reapproval is required.

Treat approval as a decision made under stated conditions, not as a permanent property of the system.

Condition Management

A mature governance record should preserve not only what was approved, but the conditions and assumptions under which the approval was reasonable. When those conditions materially change, the organization needs a defined path for reassessment.

- What assumptions supported the original approval?
- What business, user, data, technical, regulatory, or risk conditions were in scope?
- Which conditions are expected to remain stable?
- Which changes are material enough to invalidate the prior decision?
- Who determines whether reapproval is required?
- What evidence must be refreshed?



- Has the surrounding architecture changed what the system can now reach, change, or cause, even if its stated purpose and instructions remain the same?

Three Triggers for Reassessment

An AI governance decision should not be treated as permanently valid simply because it was appropriate when it was made. Organizations need defined mechanisms for determining when an existing decision should be reconsidered.

Reassessment can be triggered in three different ways:

- **Scheduled reassessment:** A planned review confirms whether the system, controls, assumptions, ownership, evidence, and operating conditions remain appropriate. Ask: Have we missed anything, and do the original conditions still hold?

Scheduled reassessment frequency should be proportionate to consequence, change velocity, autonomy, dependency, and the durability of the evidence supporting the prior decision. Lower-risk, stable uses may justify longer review intervals, while consequential or rapidly changing systems may require more frequent confirmation even when no known material trigger has occurred.

- **Recognized material change:** A known change—such as a new model, prompt, retrieval source, data source, user population, business purpose, geography, vendor, or level of autonomy—triggers reassessment based on predefined change criteria. Ask: Does this change affect the prior decision?
- **Evidence of representational non-fit:** Operational evidence suggests that real-world cases are no longer fitting cleanly within the categories, assumptions, or decision structure the governance model uses to evaluate them.

The third trigger is different from the first two. It does not require the organization to understand the new condition before recognizing that something may have changed. Instead, it treats recurring difficulty fitting operational reality into the existing governance model as evidence worth investigating.

The practitioner's responsibility is not to complete the classification. It is to represent the case accurately, including when the available classification cannot represent it.

A clean monitoring record does not necessarily prove continuing governance validity if the monitoring framework cannot represent the change that has occurred.

Detecting Representational Non-Fit

Governance processes depend on categories: risk classifications, use-case types, approval criteria, exception types, incident classifications, escalation thresholds, and other structures used to interpret operational activity.

Those categories can themselves become incomplete as AI systems and their operating environments evolve.

Organizations therefore need a legitimate way for people working within the process to indicate that a case does not fit the available categories rather than forcing it into the nearest available classification.

Potential signals include:



- repeated use of “other” or catch-all categories;
- cases repeatedly forced into categories that do not fit well;
- similar exceptions identified independently by multiple reviewers;
- increasing concentrations of non-fit around a particular category;
- recurring manual overrides or workarounds;
- cases held because existing rules do not adequately address them; and
- unusually little or no reported non-fit where the operating environment would reasonably be expected to produce ambiguity.

The objective is not simply to count exceptions but to consider pattern and shape. Several unrelated exceptions may indicate ordinary operational variation, while similar non-fit appearing independently across reviewers, categories, or time may indicate that the governance model itself is losing representational fit.

A governance system must be capable not only of detecting when it departs from the governance model, but also when reality begins to depart from what the model can represent.

Designing the Upstream Feedback Path

Detecting non-fit creates value only if the signal can influence the governance model.

Many governance processes are primarily **downstream**: policies, classifications, thresholds, and approval criteria are established and then applied to operational activity. A mature operating model also needs an **upstream path** through which operational evidence can challenge the continued adequacy of those structures.

The operating model should:

- **Make non-fit recordable.** People must have a legitimate way to indicate that an existing category or rule does not adequately represent the situation.
- **Make patterns observable.** Non-fit should be analyzed for recurrence, concentration, growth, and repetition across independent reviewers—not simply counted as individual exceptions.
- **Escalate material patterns upstream.** Defined patterns should reach an authority, warranting reconsideration of the relevant schema, policy, threshold, assumption, or governance decision.
- **Separate investigations from unilateral disposal authority where appropriate.** The schema or process owner may be best positioned to investigate a signal but should not necessarily be able to unilaterally close a material pattern of non-fit as a routine exception.
- **Make uncertainty inexpensive to record.** Recording non-fit should be at least as easy as forcing a case into an existing category. If reporting ambiguity requires substantially more effort, delay, or personal risk, the workflow will incentivize people to hide the signal.

An upstream feedback path allows operational reality to inform governance rather than allowing governance only to constrain operations.



Managing Authority State

Reassessment should not only determine whether an earlier governance decision remains valid. The operating model should also define what happens to the system's authority while validity is uncertain or when the conditions supporting that authority no longer hold.

Depending on the significance of the change, authority may remain unchanged, narrow to a more limited scope, require additional human approval, temporarily suspend, or be withdrawn until reassessment is complete.

For continuously operating or agentic systems, authority should be treated as a current operating state rather than merely as evidence that approval occurred historically (Chatterjee, 2026).

The organization should define who determines materiality, who has authority to change the system's operating authority, what evidence supports that decision, and what conditions must be satisfied before authority can be restored or expanded.

- What conditions justify the current level of authority?
- What changes are material enough to question that authority?
- Who determines whether the change is material?
- What happens to authority while reassessment occurs?
- What evidence and approval are required to restore or expand authority?

Do Not Force Uncertainty Into Certainty

A governance process needs a legitimate state for cases that cannot yet be adequately classified. When an operational case does not fit the available categories, the objective should not be to force it into the nearest category simply to complete the workflow.

A temporary state such as **Non-Fit / Pending Assessment** allows the organization to preserve the encounter while determining whether the issue is an ordinary exception, a new risk condition, or evidence that the governance model itself requires revision.

The pending state should define:

- what evidence is preserved;
- who owns the investigation;
- whether operating authority remains unchanged, narrows, or pauses;
- how long uncertainty may remain unresolved;
- what authority may close the case; and
- whether recurring cases trigger review of the underlying category, threshold, policy, or assumption.

Design principle: uncertainty should not be significantly harder to record than certainty. If practitioners are rewarded for selecting an existing category and penalized with delay or administrative burden for recording ambiguity, the operating model will gradually suppress the very signals it needs to learn.



Two Feedback Loops are Required

Operational Correction	Governance-model Correction
What should we do with this case?	Why does this type of case keep failing to fit?
Resolve an exception	Examine the structure producing recurring exceptions
Apply existing governance	Challenge whether existing governance remains adequate
May require escalation or additional evidence	May require revision of categories, thresholds, policy, decision criteria, or operating assumptions
Returns the case to operation	Changes in how future cases are evaluated

A mature operating model needs both loops. Resolving individual exceptions without examining recurring patterns can make an organization increasingly efficient at processing problems created by a governance structure that is itself becoming outdated.



12. Define the Missing Operational Ownership

Organizations often distribute AI responsibility across governance committees, product teams, engineering, legal, security, risk, and business owners. The resulting gap is frequently not a missing policy but missing coordination and operational ownership.

The AI Adoption / Operations Function

The exact job title will vary. The important capability is an accountable operational function that translates governance intent into durable ways of working and connects business adoption with ongoing control.

Capability	Typical Responsibilities
Operationalize	Translate policies and principles into workflow requirements, ownership, and routines
Coordinate	Connect business, technical, governance, risk, security, privacy, and support teams
Maintain evidence	Ensure required operational and governance evidence is produced and remains current
Monitor adoption	Identify workarounds, trust issues, review burden, training gaps, and unintended process changes
Manage change	Route material changes through the appropriate testing, documentation, and approval path
Support incidents	Ensure issues, near misses, escalations, and lessons are captured and routed
Improve	Use evidence to refine controls, workflow design, training, and scale decisions

The missing role is less about owning every control and more about ensuring the operating model continues to function across organizational boundaries.



13. Where Governance Technology Fits

Governance platforms, AI gateways, observability tools, identity systems, ticketing systems, evaluation platforms, model registries, and runtime controls can make governance enforceable and observable.

Implement shared governance capabilities at the most reusable layer possible.

As AI environments become more distributed across applications, agents, models, tools, and frameworks, organizations should consider which governance mechanisms are better implemented as shared capabilities rather than recreated independently within each AI system.

Shared capabilities may include identity and permissions, policy enforcement, evidence capture, monitoring, escalation triggers, intervention controls, and runtime authorization checks. Implementing these consistently at a platform, gateway, orchestration, identity, or other common infrastructure layer can reduce fragmentation and help prevent different systems from interpreting the same enterprise requirement differently.

This does not mean governance judgment should be centralized in technology. The enterprise operating model must still define legitimate authority, acceptable risk, evidence requirements, exceptions, escalation, and accountability. The technical layer makes those decisions consistently enforceable across systems.

Design principle: Centralize reusable enforcement where appropriate; do not centralize organizational judgment by accident.

Technology can	The organization must decide
Enforce a rule	What should the rule be, and when are exceptions permitted?
Authenticate an actor	Whether the specific action is authorized in context.
Log an event	What evidence is sufficient, and how long does it remain relevant?
Block or pause an action	Who has authority to release, override, or escalate?
Monitor thresholds	Which thresholds matter and who owns response?
Record approval	What was approved, under what conditions, and when it expires?
Route workflow	Who owns the decision and consequences?

Keep a clean interface between technical enforcement and organizational authority.

From Organization Decision to Runtime Enforcement

The operating model and runtime architecture perform different but complementary functions. Technical systems can detect conditions, enforce constraints, validate permissions, preserve evidence, and prevent or allow execution. The organization must determine what those signals mean: what constitutes material change, what authority is appropriate, what evidence is sufficient, who may make the decision, and under what conditions authority can be restored.

The objective is to make organizational governance technically enforceable without defaulting to converting organizational judgment into technical logic.

Runtime/ technical layer	Enterprise operating model
Detects changed conditions	Defines which changes are material
Evaluates observable state	Defines acceptable operating conditions
Enforces authority state	Determines appropriate authority
Blocks or permits execution	Defines who may authorize exceptions
Produces runtime evidence	Defines evidence sufficiency
Detects threshold breach	Defines escalation consequence
Enforces restored authority	Defines restoration criteria

Verification does not create decision authority.

A runtime control may establish that identity, authority state, policy, evidence, scope, and other required conditions appear to be satisfied. The operating model must still define what component or decision authority has standing to resolve those conditions into permission to create consequence. For consequential actions, that authority should be explicit rather than inferred from successful verification or from the absence of a detected invalidator.

Practitioner Question

What has authority to convert verified runtime conditions into a binding decision to permit or refuse consequence?

Pre-commit Control Principle

For consequential actions, runtime governance should distinguish between controls that observe or record activity and controls that can prevent the action from committing. Evidence, logging, and provenance remain important, but they are not substitutes for a control that can hold, refuse, or escalate the action before consequence occurs. Where pre-commit intervention is required, architecture should establish a resolved identity, current authority, applicable approval, and an enforceable pending-or-refusal state before execution proceeds.

Runtime Hold Test

Where consequential execution depends on approval or intervention, practitioners should establish:

- **Pause layer** — At what technical layer is execution held, and can the action route around that hold?
- **Hold duration** — How long can the action remain pending before the system requires another disposition?
- **Timeout behavior** — If the required decision is not received, does the system deny, remain pending, escalate, degrade to another mode, or proceed?

Governance Continuity Principle

A runtime governance determination should remain binding through the path that produces consequence. For consequential actions, verify not only that the action passed an admissibility decision, but that the resulting constraints remain attached from determination through commit and execution. If an alternate



route can reach the same consequential outcome without preserving equivalent constraints, the governance decision is not fully enforced.

Practitioner Question:

Once a runtime governance decision is made, what ensures that the decision and its constraints remain binding until the consequential action executes?

When several authorities or controls must approve a consequential action, can the organization establish that each determination applies to the same specific action, scope, and destination, and that all remain valid at execution time?

Technical architecture can also show that the governance model struggles to represent operational reality. Systems may surface unclassified events, conflicting signals, recurring exceptions, failed rule matches, unusual concentrations of overrides, or conditions the existing schema cannot evaluate. The enterprise operating model must determine what those signals mean, who investigates them, what happens to operating authority while uncertainty persists, and when recurring non-fit warrants revising the governance model itself.

Approval scope should be sufficiently specific that materially different actions require new authorization. AI reasoning should not automatically imply execution authority.

Authority restoration should require explicit evidence that the conditions supporting renewed authority have been satisfied (Chatterjee, 2026).

Route Closure Question:

Can the same consequential outcome be reached through another route that bypasses equivalent governance controls?

Multi-step agent governance

In agentic workflows, validating each individual action may not be sufficient. A sequence of locally admissible actions can collectively move beyond the approved purpose, authority, risk tolerance, cost boundary, or intended outcome.

Runtime governance should therefore consider two levels of evaluation:

- **Local admissibility** — Is this specific proposed action admissible now under the current authority, evidence, policy, risk, constraints, and approval conditions, even if the agent itself remains generally authorized to operate?
- **Workflow trajectory** — Does the evolving sequence of actions remain within the approved operating envelope for the workflow as a whole?
The operating model should define which cumulative changes are material enough to trigger escalation, additional approval, narrowed authority, pause, or termination.

Agentic runtime question: Can every individual action remain technically permissible while the workflow becomes unacceptable?

14. Measure Whether the Operating Model Works

A mature AI operating model measures both AI outcomes and the health of the governance process itself.



Dimension	Illustrative Measures
Value	Cycle time, cost, throughput, customer outcome, quality improvement
Quality	Correction rate, acceptance rate, rework, failure rate
Risk	Incidents, near misses, threshold breaches, sensitive-output events
Adoption	Usage, satisfaction, workarounds, abandonment, training need
Review burden	Time spent verifying, correction effort, escalation effort
Governance throughput	Time to classify, review, approve, reassess, or close exceptions.
Evidence health	Missing/expired records, stale owners, incomplete decision rationale, version mismatch, observation coverage: governance questions for which no reliable observation channel exists; missing evidence contracts; material decisions that cannot be reconstructed from retained evidence.
Change control	Material changes identified, reassessments completed, unauthorized changes
Governance fit	Non-fit records, concentration by category, recurring non-fit across independent reviewers, trend/growth over time, forced-fit classifications, “other” usage, manual workarounds/overrides, material patterns escalated upstream, patterns closed as exceptions, absence of non-fit where reporting channels exist, schema/policy revisions resulting from operational evidence.
Ownership	Unassigned controls, overdue actions, unresolved escalations
Continuity	Fallback test results, recovery time, retained human capability
Control coverage	Percentage of in-scope consequential actions that actually transit the intended governance control path; unexplained gaps between the population of relevant actions and the population observed by the control.
Override/bypass use	Frequency, scope, duration, ownership, and recurrence of break-glass or override paths; whether temporary bypasses are becoming routine operating paths.
Cross-functional disagreement	Recurring material differences among business, engineering, security, privacy, legal, risk, Responsible AI, or other decision functions about whether a use case is ready or appropriately controlled.
Unexpected governance silence	Usually, clean reporting in an environment where ambiguity, exceptions, or boundary cases are reasonably expected prompts examination of whether the reporting structure can surface uncertainty.

Do not measure only how fast AI produces output. Measure the human effort required to trust, verify, correct, govern, and sustain it.

Disagreement is not always governance failure. Sometimes disagreement is governance evidence. Before resolving conflicting assessments, determine what the disagreement reveals about evidence sufficiency, risk assumptions, decision criteria, scope, or authority.

Runtime authority question

Before an AI system or agent acts, can the organization determine not only whether the actor or system is authenticated, but whether this specific action is authorized under the current conditions?

What important governance question could we not answer today because no evidence-producing mechanism exists?



15. Turn Analysis into Executive Decisions

Executives generally do not need the full practitioner workbook. They need a clear recommendation, the rationale behind it, the governance model, the evidence plan, the pilot recommendation, and the decision required.

Briefing Element	Executive Question
Business problem	What problem are we solving?
Recommended opportunity	What are we proposing and why this?
Future-state design	How will work change?
Governance model	Who reviews, approves, escalates, intervenes, and remains accountable?
Evidence plan	How will we know whether it creates value safely?
Pilot recommendation	What are we testing and learning?
Risks/dependencies	What could block or change the recommendation?
Decision requested	What do you need me to approve, fund, sponsor, pause, or decide?

Executive briefings should end with a decision, not a general discussion. It is a concise decision document, not a summary of all practitioner analysis.



16. Practitioner Conversation Guide

The questions below are intended as facilitation prompts rather than a questionnaire to be handed to stakeholders all at once.

With a Business Owner

- Walk me through the work as it happens today.
- Where is the friction, waiting, rework, or review burden?
- What outcome would make this worth changing?
- What happens if the AI output is wrong?
- What decisions must remain human?
- Who owns the outcome after launch?

With Engineering / Architecture

- What models, prompts, retrieval sources, tools, APIs, and systems do this depend on?
- What is versioned today?
- What can change without a formal release?
- What telemetry and logs exist?
- What controls can be enforced at runtime?
- Which governance controls are being implemented separately within individual AI systems, agents, or frameworks that could instead be provided consistently as a shared capability?
- What happens when a dependency fails?
- Can we distinguish authentication, authorization, enforcement, and evidence-sealing events in the runtime record, especially across agent or system handoffs?
- At what technical layer can the action be paused, blocked, or held for review, and what happens when no enforceable intervention point exists?
- If the system cannot hold execution, does the control degrade to notify-and-continue, and is that behavior explicitly reflected in the governance model?
- Can the system hold or refuse a consequential action before committing, or can it only observe, notify, or reconstruct the action afterward?
- If approval is required, can the action remain in a true pending state until that approval is resolved?
- Can the system discover, invoke, or spawn tools, agents, permissions, or downstream capabilities that were not explicitly evaluated and authorized?
- How does the system evaluate both the admissibility of individual agent actions and whether the cumulative workflow remains within its approved authority, risk, cost, and purpose boundaries?
- Which actions require synchronous governance checks, and which conditions can be monitored across the workflow without introducing unacceptable latency?
- Which sources and contextual inputs can the AI treat as authoritative, and how does the system respond when those sources are stale, conflicting, incomplete, or unavailable?
- Which controls can be applied earlier in the lifecycle so that an invalid condition is prevented from advancing rather than detected only after execution or deployment?
- Once a runtime governance decision is made, what ensures that the decision and its constraints remain binding until the consequential action executes?



- What has authority to convert verified runtime conditions into a binding decision to permit or refuse consequences?

With Governance / Legal / Risk / Security

- What decisions require review or approval?
- What evidence do you need to make those decisions?
- What assumptions or conditions are attached to approval?
- What changes should trigger reassessment?
- Who can accept risk, grant exceptions, pause use, or require remediation?
- What must be preserved if an incident occurs?
- Who determines which sources are authoritative for consequential decisions, and what level of uncertainty requires escalation or non-decision?

With Frontline Users and Managers

- How are you using the system?
- What do you verify before trusting an output?
- Where does it save time, and where does it create new work?
- What do you do when it is wrong?
- What workarounds have emerged?
- Would you know how to report a near miss or unsafe result?



17. Operational Red Flags

If You Hear...	Investigate...
"Legal approved it."	What exactly was approved, when, under what conditions, and whether the system has changed
"A human reviews it."	Reviewer, criteria, authority, frequency, evidence, escalation
"A human has to approve high-risk actions."	Investigate: Where does execution actually pause? Can the architecture hold the action before consequence, or does it only notify after execution has begun?
"Only Alex/Jordan/etc. really knows how this works."	Investigate: undocumented decision logic, hidden dependencies, tacit exception handling, ownership gaps, evidence gaps, and whether the operating model can survive the loss of the key individual.
"We have logs."	Integrity, provenance, access, version alignment, decision relevance
"The model is monitored."	Owner, thresholds, cadence, response path, business context
"It passed the pilot."	Exit criteria, evidence, operating-model performance, scale decision
"Security owns AI governance."	Business accountability, decision rights, cross-functional ownership
"The vendor handles that."	Retained enterprise accountability and vendor dependency
"We can always do it manually."	Whether the manual process still exists, is documented, staffed, and tested
"Nothing material changed."	Prompt, retrieval, data, user, purpose, autonomy, vendor, and business-condition changes
"Everyone owns it."	Whether anyone has explicit accountability and intervention authority
"Just choose whichever category is the closest."	The category system used to approve. It may no longer be able to flag something that invalidates the approval process. If the selected category is the closest fit, it doesn't require the organization to understand the new condition or detect if something is wrong.
"We have almost no non-fit cases."	Zero residue may mean excellent fit, or it may mean you've built no viable way to record non-fit.
"We only use the bypass in emergencies."	Investigate: who owns it, how often it is used, whether it expires, whether every invocation is recorded, and whether some in-scope actions can bypass the control without invoking the formal override.



18. Practitioner Deliverables

The existing AI Operational Readiness Kit remains the working-tool layer supporting this guide.

Deliverable	Purpose
Discovery Workbook	Capture current-state observations across people, process, technology, governance, measurement, adoption, continuity, escalation, and ownership
Opportunity Prioritization Matrix	Rank opportunities using value, fit, complexity, risk, and continuity considerations
AI Use Case Categories	Translate business problems into repeatable AI patterns
Current-State → Future-State Framework	Design how work changes with AI assistance and human accountability
Decision Rights Framework	Define review, approval, escalation, intervention, and accountability
Evidence Framework	Define value, quality, risk, adoption, decision, and governance evidence
Pilot Readiness Framework	Determine whether the operating model is ready to test
Executive Briefing	Turn analysis into an executive decision
Governance Operating Rhythm	Define monitoring, evidence maintenance, change, incident, adoption, and lifecycle routines
Change Trigger & Reapproval Matrix	Define what changes require documentation, testing, review, escalation, or reapproval
Governance Evidence Register	Maintain decision provenance, conditions, owners, versions, and evidence integrity

The guide explains the method. The toolkit helps practitioners perform the work.

Change Trigger & Reapproval Matrix Example:

Trigger	How detected	Who evaluates it	Immediate authority state	Required Review	Evidence refreshed	Reapproval?
Model version change	Release/change process	Product + Risk	Usually, unchanged pending review	Regression/ risk testing	Model/ version tests	Depends on materiality
New user population	Business request	Business + Governance	May narrow	Access/training/ risk review	User/risk evidence	Possibly
New tool/ integration	Architecture change	Architect + Governance	May narrow/ pause	Reachability/ control review	Permission, tests	Often
Incident/ near miss	Incident process	Incident authority	May pause	Control reassessment	Incident/ remediation evidence	Often
Repeated non-fit	Governance monitoring	Governance/ schema owner	Depends on consequence	Investigate model fit	Non-fit cases/ assumptions	Possibly



19. Worked Governance Example: Meeting Summary Assistant

This deliberately simple use case illustrates how the methodology connects across stages.

Stage	Example
Problem	Managers spend significant time preparing meeting summaries and action items.
Opportunity	Reduce preparation effort and improve follow-up consistency.
AI pattern	Summarization + extraction.
Future state	AI drafts summary and action items; manager verifies before distribution.
Decision rights	Manager reviews and approves; sensitive topics escalate; manager may disable AI assistance; manager remains accountable.
Evidence	Preparation time, correction rate, sensitive-topic escalations, usage, satisfaction.
Pilot	30-day internal pilot with defined scope, guidance, support, and manual fallback.
Operate	Monitor correction rate, complaints, sensitive-topic handling, adoption, and review burden.
Change trigger	New transcript source, external distribution, new user population, prompt change, or use for high-consequence meetings triggers reassessment.
Lifecycle decision	Scale only if value is demonstrated, quality thresholds hold, review burden remains acceptable, and governance controls work in practice.



20. A Simple Maturity Lens

Level	Characteristics
1. Experimenting	AI use is local; controls are informal; ownership and evidence are inconsistent
2. Defined	Policies, intake, risk classification, and approval processes exist
3. Operational	Decision rights, evidence, monitoring, support, escalation, and ownership are embedded in workflows
4. Adaptive	Material changes trigger reassessment; evidence remains aligned to system state; governance drift is actively managed
5. Learning	Evidence drives continuous improvement, portfolio decisions, control refinement, and responsible scaling

The objective is not governance maturity for its own sake. It is the ability to create value while preserving accountability as AI-enabled work changes.



Appendix A. Engagement Checklist

- Obtain strategy, policies, standards, AI inventory, pilot list, architecture, risk taxonomy, incident process, and current governance artifacts.
- Identify sponsors, business/process owners, governance stakeholders, technical owners, users, and support teams.
- Select representative use cases to trace end-to-end.
- Map current-state workflow and dependencies.
- Identify AI patterns and intended value.
- Define future-state workflow and fallback.
- Assign decision rights and intervention authority.
- Define evidence and measurement plan.
- Confirm pilot readiness.
- Define monitoring ownership and governance operating rhythm.
- Define material-change triggers and reapproval rules.
- Confirm incident, continuity, and retirement paths.
- Prepare executive recommendations and explicit decision request.

Appendix B. Core Principles

- Start with work, not tools.
- Policy is not the operating model.
- Not every opportunity should become an AI project.
- AI-enabled work should be designed before it is deployed.
- Human involvement does not equal human accountability.
- Authority should exist before it is needed.
- Observable systems are governable systems.
- Governance decisions should rely on evidence, not optimism.
- A pilot should test the operating model, not only the technology.
- Deployment is not the end of governance.
- Monitoring without ownership is observation, not control.
- Approval is conditional on the circumstances under which it was granted.
- Evidence must remain aligned to the system actually operating.
- Technology enables governance; organizational authority defines it.
- When multiple AI systems require the same governance control, prefer a shared enforcement capability where feasible rather than independently recreating the control in every system.
- Organizations should scale learning, not assumptions.
- The purpose of the methodology is not documentation. It is better decisions.



Appendix C. Development Notes & Practitioner Acknowledgments

This guide consolidates the June 2026 AI Operational Readiness Kit with later practitioner insights developed through discussion of operational readiness, evidence management, operating models, decision rights, governance drift, condition management, runtime governance, continuity, and post-deployment accountability.

Practitioner dialogue and conceptual influences

This methodology is independently developed by Chloe C. Devine. Its evolution has also benefited from substantive exchanges with practitioners working on adjacent problems in governance, evidence, runtime, and technical architecture. These discussions helped challenge, refine, or extend aspects of the operating model without transferring ownership of the overall methodology.

Ahmed Al-Saedi — Practitioner discussions helped sharpen the treatment of continuing authority in changing AI systems, including the distinction between historical validity and current operational authority; the need to preserve the rationale, assumptions, and validity conditions supporting a governance decision; condition-aware rather than purely calendar-based reassessment; governed and versioned transition criteria; and the importance of a structured challenge path that allows credible uncertainty to trigger review before a formal control failure or incident occurs.

Aleksandra Pinar — Practitioner discussion helped sharpen the distinction between an AI system's continuing authority to operate and the admissibility of a specific proposed action at execution time. The exchange reinforced that an agent may remain generally authorized while a particular decision becomes inadmissible because current evidence, risk, constraints, context, or approval conditions have changed. This informed the treatment of decision-specific runtime admissibility and the use of PASS, HOLD, ESCALATE, or REJECT as properties of the action being evaluated rather than as blanket properties of the system or agent.

Antonio Silva Perez — Practitioner discussion helped sharpen the treatment of runtime escalation and signal-to-action governance, particularly the need to connect monitoring signals to defined routing, evaluation, decision authority, intervention, and evidence preservation. The exchange reinforced that detection alone is not an operational control unless the organization has an explicit path for determining what the signal means, who has authority to respond, what action is expected, and how the resulting decision and intervention are preserved.

Atul Yadav — Practitioner discussion helped sharpen the treatment of shared governance capabilities, particularly the value of implementing reusable controls such as identity, permissions, policy enforcement, evidence capture, monitoring, escalation, and runtime authorization at a common platform or orchestration layer rather than recreating them independently across individual AI systems or agent frameworks. The exchange also helped reinforce the distinction between centralized technical enforcement and organizational responsibility for defining authority, acceptable risk, exceptions, evidence requirements, and accountability.

Christof Schumann — Practitioner discussion helped sharpen the treatment of temporal separation in runtime governance, particularly the need to distinguish authentication, authorization, enforcement, and evidence-sealing events rather than collapsing them into a single activity timestamp. The exchange also reinforced the importance of preserving current authority across delegated or multi-agent handoffs and maintaining sufficient separation between execution, legitimacy determination, and evidence custody. The



exchange also helped sharpen the distinction between verifying that runtime conditions are satisfied and possessing the decision authority to convert those verified conditions into permission to create consequence.

Connie Yang — AIGovsOps: FailFestLIVE!, AI House, Seattle, WA, September 3, 2026 — A live practitioner discussion helped sharpen the treatment of source and context authority, particularly the risk that an AI system can reason coherently from information that is inaccurate, stale, incomplete, or otherwise unsuitable as a basis for decision-making. The discussion clarified the distinction between what information a system can access and what sources, context, assumptions, or system state it should treat as sufficiently authoritative when forming an output or action.

Deepak K. Eidnani — Practitioner discussion helped sharpen the treatment of multi-step agent governance, particularly the distinction between validating individual actions locally and evaluating whether the evolving workflow remains within its approved purpose, authority, risk, cost, and operating boundaries. The exchange also highlighted the need to design runtime governance proportionately so that effective control does not become an unnecessary operational or latency bottleneck.

Femi Idowu — Practitioner discussions helped sharpen the treatment of the interface between enterprise governance and technical implementation, particularly the sequencing problem that arises when organizations implement tools or controls before establishing decision rights, acceptable risk, evidence requirements, and escalation logic. The exchange also informed the distinction between principle-based governance requirements and the specific, testable operational decisions required for technical enforcement; the feedback loop between governance, implementation, monitoring, and adjustment; and the use of recurring operational signals to determine when the governance design itself may require reassessment rather than simply processing another exception.

Ivan Leonel Freitas Sousa — Practitioner discussions and his work on runtime governance (*FIN: Four Interdependent Nodes* (2026)) and *Registration Without Representation: Scope Validity and the Blind Field of AI Governance* (2026) helped sharpen the distinction between historical approval and continuing authority at execution time, including authority and evidence freshness, materiality evaluation, authority-state change and restoration, decision provenance, and the need to consider whether equivalent execution paths can bypass intended governance controls. These concepts have been translated here into enterprise operating-model practices; this guide does not adopt or reproduce the FIN architecture.

Jay Smith — Practitioner discussion helped sharpen the treatment of action-specific authorization in multi-authority workflows, particularly the need to preserve the scope and current standing of multiple required determinations and bind them to the specific consequential action being executed. The exchange also reinforced the value of fail-closed behavior when required authority cannot be established, so that unresolved uncertainty does not become implicit permission to proceed.

Jeremy Gomsrud — Practitioner discussion helped sharpen the treatment of runtime intervention feasibility, particularly the need to verify where in the technical stack a consequential action can be paused, blocked, or held for human review. The exchange highlighted the risk that a policy requirement for human approval may degrade in implementation to notify-and-continue when no enforceable pause point exists and further clarified that runtime hold behavior should explicitly define the pause layer, maximum hold duration, and default action when approval is not received. This reinforced the need to treat timeout and non-response behavior as part of the governance model rather than as an implementation detail.

Michael Kaplan — Practitioner discussion helped sharpen the treatment of reconstructable decision records, particularly the need to preserve the initiating signal, relevant context and evidence, authority



invoked, model and tool versions, proposed action, applicable gate or approval condition, approval or override, execution, and resulting state change as a connected record. The exchange also reinforced the distinction between continued permission and continued legitimacy, including the need to re-establish current authority and evidence at consequential boundaries rather than treating the absence of revocation as sufficient reason to proceed.

Patrick Juvet Etoua — Builder of FactNotebook and author of *Governance Falsifiability Infrastructure*. Practitioner discussions and review of FactNotebook’s published evidence-reconstruction work helped sharpen the treatment of reconstructability as a lifecycle governance capability; the timing and ownership gap between evidence creation and later accountability; the distinction between reconstructing what evidence can establish and the assurance judgment applied to it; observation gaps and missing evidence channels; and the principle that evidence requirements should be designed around the governance questions future decision-makers may need to answer.

Paul Kanaris — Practitioner discussion helped sharpen the distinction between legitimate execution and decision quality, particularly the possibility that an AI-enabled action may be properly authorized, satisfy policy and control requirements, and still rest on incomplete context, flawed assumptions, conflicting objectives, or weak reasoning. The exchange informed the treatment of decision-basis sufficiency and the need to scale decision-quality assurance with consequence and risk.

Ryan Feller — Practitioner discussion helped sharpen the treatment of temporal evidence and functional separation in runtime governance, particularly the distinction among authorization, monitoring, evidence preservation, and provenance as mechanisms that may surround the same event but establish different facts. The exchange also highlighted the need to preserve enough bounded historical state to reconstruct what policy, authority, evidence, context, and system conditions were in force at the time of a consequential action, without projecting later conditions or interpretations backward onto the event.

Sudip Chatterjee — Practitioner discussions and his published work on runtime governance for agentic AI helped sharpen the treatment of authority and evidence freshness, runtime admissibility, authority-state transitions and restoration, decision provenance, execution boundaries, governance continuity from determination through commit and execution, and route closure. The exchange also helped clarify the interface between technical detection and enforcement and the enterprise operating model’s responsibility for defining materiality, evidence sufficiency, decision authority, and restoration criteria.

Saurabh Singh — Practitioner discussion helped sharpen the treatment of runtime decision sufficiency, particularly the distinction between whether an AI system or agent is authorized to act and whether the evidence and context supporting the proposed action remain sufficiently current, complete, applicable, and reliable at the moment of execution. The exchange reinforced the need to treat authority and decision basis as separate runtime questions and helped clarify the role of evidence freshness in determining whether an otherwise authorized action should proceed.

Thomas Bronack — Practitioner discussion and review of the DCAG System executive overview helped sharpen the treatment of lifecycle-stage control, upstream enforcement, and evidence-producing governance decisions. The DCAG architecture illustrates a recurring control loop spanning planning, engineering, development, production acceptance, deployment, monitoring, and continuous authorization. The exchange also reinforced the value of defining explicit advancement conditions at lifecycle stages and preserving evidence when an action or artifact is blocked, held, or refused rather than only when execution succeeds.



Tom Staskiewicz — Practitioner discussion helped sharpen the treatment of agentic capability reachability, particularly the need to evaluate not only what an AI system is intended to do, but what downstream tools, agents, permissions, dependencies, and consequential actions the surrounding architecture makes reachable. The exchange reinforced that runtime authority can be shaped by technical reachability; that reachable capability should be bounded and reassessed throughout the lifecycle as the architecture changes; and that where every possible execution path cannot be known in advance, governance should also limit the consequence that an unanticipated path can create.

Unnamed participant — AIGovsOps: FailFestLIVE!, AI House, Seattle, WA, September 3, 2026 — A participant raised the “hit by a bus” problem as an analogy for key-person dependency in AI operations: if critical knowledge, judgment, system assumptions, or exception handling resides only with one individual, the organization has not yet converted that expertise into a durable operational capability. The discussion helped sharpen how we treat continuity, institutional knowledge, and resilience in AI operating models.

Vipul Patel — Practitioner discussion helped sharpen the distinction between runtime observation and pre-commit governance, particularly the need for consequential actions to be enforceable under resolved identity and current authority before they commit. The exchange highlighted that logging, provenance, and post-action reconstruction are not equivalent to a control that can hold, refuse, or escalate execution, and reinforced the importance of governing the decision that produces consequence rather than only the record produced afterward.

References

This section is for published work, formal frameworks, papers, technical architectures, standards, and public sources that materially informed this guide.

Bronack, T. (2026, August 26). *The DCAG System: One control architecture. Four factories. Continuous Authorization to Operate* [Partnership briefing]. Data Center Assistance Group, LLC.

<https://assets.zyrosite.com/YleW0l1LaPt59Qa3/dcag-system---executive-overview-8-26-2026-y4wLm2QhxsWNIim.pdf>

Chatterjee, S. (2026, August 28). *Agentic AI governance cannot stop at the decision. It must control the consequence.* LinkedIn.

<https://www.linkedin.com/pulse/agentic-ai-governance-cannot-stop-decision-must-sudip-chatterjee-n9qxf>

FactNotebook. (2026). *Evidence Reconstruction Dossier: Governance observability of a NeoMundi ControlTower pilot — 5 governed GPT-4o generations on PubMedQA.* FactNotebook.

[Evidence Reconstruction Dossier](#)

- **Methodology paper:** DOI 10.5281/zenodo.20764196
- **Output-grammar specification:** DOI 10.5281/zenodo.21213203
- **Evidence Attributes specification:** DOI 10.5281/zenodo.20709010

Sousa, I. (2026). *FIN: Four Interdependent Nodes: A primer* (Version 1.1). Zenodo.

<https://doi.org/10.5281/zenodo.22261891>

Sousa, I. (2026). *Registration Without Representation: Scope Validity and the Blind Field of AI Governance.*

Zenodo. <https://doi.org/10.5281/zenodo.21458848>



Future Research and Validation

- Does every lifecycle stage have a clear entry condition, activity, output, owner, evidence requirement, and exit decision?
- Where should risk tiering enter the methodology, and when should risk classification be reassessed?
- How should regulatory obligations be mapped to operating decisions, evidence, and lifecycle stages without turning the Guide into a compliance manual?
- What minimum evidence should be universal, and what evidence requirements should vary by risk tier or consequence?
- What constitutes a material change for different AI patterns and levels of autonomy?
- How should agentic workflows alter authority, runtime control, intervention, and evidence requirements?
- What governance capabilities should be centralized at the enterprise level, and what should remain federated to business or product owners?
- What operating metrics best reveal governance drift before an incident or control failure occurs?
- What role should automated governance tooling play at each lifecycle stage, and where should human judgment remain explicit?
- What practitioner artifacts are essential to operationalize the methodology, and which formats best support their use?
- Which governance properties must remain invariant as agentic systems, execution paths, tools, and handoffs become more dynamic?

